Discrete Deformation Calculus

A Finite Trace Calculus for Obstruction Handling

Jeremy H. Carroll

Version 1.0.3 — 2026-10-03

Text: CC BY 4.0 — Code: PolyForm Noncommercial 1.0.0

Abstract

The Discrete Deformation Calculus (DDC) records what a finite computation did to a declared obstruction. Each step emits one letter from $\mathcal{A}_{\mathrm{DDC}} = \{E, R, C, I, \Lambda, D, M, T\}$: exact repair, residual projection, context exposure, integration, licensed lift, descent, typed refusal and terminalization. A step carries a receipt, and a tape is the hash-chained sequence of receipts. Diagnostics come from CODA-FOC: Φ_1 is the weighted ℓ^1 distance of a residual to the image of d_0, and $\Gamma_2 = \|d_1 x\|_1$ is its face-closure defect.

This version states the letter laws as a finite-control transition system over unbounded data and proves its tape results in Lean 4 without Mathlib. Every proved or computed claim is bound to a Lean declaration, a Python test or a written proof shipped in this package; the rest are typed as open problems or positioning. The main results are these. Only a descent with a loss ledger or a licensed lift can lower Φ_1 or Γ_2. Consequently an open obstruction ($\Gamma_2 > 0$) never reaches a terminal without such a priced move. The tape $\mathrm{C} \cdot \mathrm{R} \cdot \mathrm{E} \cdot \mathrm{D} \cdot \mathrm{I} \cdot \mathrm{T}$ is lawful exactly when each of its six steps satisfies its law. A lawful one starts from $\Phi_1 = \Gamma_2 = 0$, and without a loss ledger at D it is not lawful. Every step the paper admits satisfies the deposited historical comparison laws on the same abstract state; no deployed-runtime equivalence is claimed. A lawful tape is replay evidence; it is never authority.

0. Scope and nonclaims

```
DDC does not prove that reality is discrete or that the universe is a cochain complex.
DDC does not prove quantum mechanics, continuum physics, consciousness, semantic truth,
autonomous self-modification, or global topology.

DDC is not authority. A DDC tape is replay evidence; a receipt is bounded verification
data; a terminal is a routing terminal. No authorization is conferred.

"All problems are obstruction problems" is a programme thesis, not a theorem.
"The universe is not very creative" is motivation, not runtime mathematics.
```

Every diagnostic in this paper is taken on a declared finite carrier with a declared detector. Claims about trace equivalence, normal forms, detector adequacy in general, and lift minimality remain open (§13).

Part I — The calculus

1. Definition

Definition 1.1 (OE010-09). A *DDC regime* is a finite carrier, declared coboundary maps d_0, d_1 with $d_1 d_0 = 0$, a detector that measures Φ_1 and Γ_2, and a terminal policy. A *step* is a letter from $\mathcal{A}_{\mathrm{DDC}}$, the state after it, and the witnesses it carries. A *tape* is a finite sequence of steps; it is *lawful* when every step satisfies its letter's step law (§7) and the control laws (§10).

Carrier, ZeroRel, Observer, Witness, Residual, Quotient, CompletionLaw and AuthorityEffect are used in their CODA sense; this paper relies only on the ones it defines in this section and in §3 (OE010-02).

The letters classify the role a step played in the life of an obstruction:

```
context exposure · exact repair · residual projection · integration
licensed lift · descent · typed refusal · terminalization
```

2. Core invariant

DDC does not only classify obstruction states. It classifies what each computation did to an obstruction:

> DDC is a typed calculus of what a computation did to an obstruction.

A status system says *pass*, *fail* or *unknown*. DDC asks which finite operation occurred: did it expose a carrier, repair a defect, project a residual, integrate a receipt, license a lift, descend with loss, refuse, or terminalize? This lets error handling be compared across domains without claiming that the domains share an ontology.

3. Cochain window and diagnostics

When a cochain regime is declared, every execution window is

$$C^0 \xrightarrow{d_0} C^1 \xrightarrow{d_1} C^2, \qquad d_1 \circ d_0 = 0.$$

Object	Runtime reading
C^0	vertex variables, potentials, repair parameters
C^1	edge defects, claim gaps, local residuals
C^2	face and loop closure defects

The convention is adapter-conditional: a prompt, a patch or an agent trace does not have $C^\bullet$, d_0, d_1, Φ_1 or Γ_2 until a carrier, a comparison map, a repair image and a detector have been declared.

Definition 3.1 (FOC diagnostics). For a residual $x \in C^1$ with positive edge weights w, $\Phi_1(x) = \min_\varphi \sum_e w_e |x_e - (d_0\varphi)_e|$ and $\Gamma_2(x) = \|d_1 x\|_1$. The class of x is *ActiveClosureFailure* when $\Gamma_2 > 0$,

otherwise *ExactRepair* when $\Phi_1 = 0$, otherwise *ClosedResidual*. A positive Γ_2 dominates (witness dominance).

Proposition 3.2 (DDC-TAPE-11). On a finite graph, Φ_1 is attained at a potential determined by a spanning forest whose edges are all tight. Enumerating spanning forests therefore computes Φ_1 exactly.

Proof. Fix one vertex potential to zero in each connected component. Positive weights bound each edge difference on an objective sublevel set; paths in a spanning forest then bound every remaining potential. Apply compactness to the real extension; a minimum is attained. The objective is piecewise linear on finitely many polyhedral regions. The minimizing face in one such region is nonempty and bounded, so it contains a vertex of that region. At such a vertex, independent tight constraints $x_e = \varphi_v - \varphi_u$ fix φ. Independent incidence rows form a forest, and fixing φ needs one spanning each component. A minimizing vertex determined by a spanning forest has rational coordinates, so the optimum is also attained over the rationals. □

`cochain.phi1` implements the enumeration and refuses as `BUDGET_UNKNOWN` above a declared subset budget; `test_cochain.py::test_phi1_is_gauge_invariant_and_matches_brute_force_on_small_grids` checks it against an integer search. `cochain.phi1_certificate` also returns a dual witness: a flow y with $d_0^\top y = 0$ and $|y_e| \le w_e$ whose value $\langle y, x\rangle$ equals the primal objective. Dual vertices set $y_e = \pm w_e$ off a spanning forest and solve the forest edges by conservation, so the same enumeration finds them, and strong duality is checked rather than assumed (`test_cochain.py::test_phi1_certificates_close_the_duality_gap_exactly`). Every Φ_1 in the demos is measured through this certificate.

A fixed forest representative need not minimize this objective. For the complete graph on $a < b < c < d$, orient all six edges increasingly and put -1 on each. The forest rooted at a leaves the three other edges at -1, so the recorded residual norm is 3, while $\Phi_1 = 2$. Both representatives have the same quotient class. Chapter 2 distinguishes these quantities.

Theorem 3.3 (OE010-01, DDC-TAPE-10). Over $\mathbb{Q}$ with the standard inner product, $C^1 = \operatorname{im} d_0 \oplus \operatorname{im} d_1^\top \oplus \mathcal{H}^1$ with $\mathcal{H}^1 = \ker d_1 \cap \ker d_0^\top$. Write $x = x_E + x_C + x_H$. Then $\Gamma_2(x) > 0$ exactly when $x_C \neq 0$, and $\Phi_1(x) = 0$ exactly when $x_C = 0$ and $x_H = 0$.

Proof. Set $U = \operatorname{im} d_0$ and $V = \operatorname{im} d_1^\top$. The chain law gives $U \perp V$, and $(U \oplus V)^\perp = \ker d_0^\top \cap \ker d_1 = \mathcal{H}^1$. Finite-dimensional positive definiteness gives the unique orthogonal direct sum $C^1 = U \oplus V \oplus \mathcal{H}^1$. The defining linear systems have rational coefficients, so their unique projected vectors are rational. Now d_1 kills x_E because $d_1 d_0 = 0$ and kills x_H by definition. On $\operatorname{im} d_1^\top$ it is injective: $d_1 d_1^\top \psi = 0$ implies $\|d_1^\top \psi\|^2 = \langle \psi, d_1 d_1^\top \psi\rangle = 0$. So $d_1 x = d_1 x_C$ vanishes only when $x_C = 0$. $\Phi_1(x) = 0$ means $x \in \operatorname{im} d_0$, which by the direct sum means $x_C = x_H = 0$. □

The subscripts name Hodge components, not letters. The letter E acts on the exact component; the coexact component is what Γ_2 sees; the harmonic component is the closed residual. `cochain.hodge_split` computes the split exactly and `test_cochain.py::test_hodge_split_is_orthogonal_and_matches_the_classes` checks the theorem on four complexes.

4. The alphabet

Every state transition or structural modification in a DDC regime emits exactly one letter.

Letter	Name	Runtime meaning	Must not claim
`E`	Exact repair	removes a defect that lies in $\operatorname{im} d_0$	semantic truth, global correctness, authority
`R`	Residual projection	records the residual class; keeps Φ_1 and Γ_2	resolution
`C`	Context exposure	declares or splits the carrier, window or boundary	repair or authority
`I`	Integration	appends a receipt to the hash-chained tape	promotion
Λ	Structural lift	extends the carrier under a licence	arbitrary growth or automatic promotion
`D`	Descent	re-indexes or coarsens with a declared loss ledger	lossless transport unless witnessed
`M`	Typed refusal	refuses an invalid, unwitnessed or out-of-budget request	failure of the product
`T`	Terminalization	closes the tape on a routing terminal	authority

`DDC records how an obstruction was handled, not whether the world is true.`

5. Step receipts

Every admitted abstract step produces a deterministic record of the supplied states, carrier labels and witness flags, linked to the previous record. These flags are hypotheses about evidence presence; they do not authenticate a receipt, a loss ledger, a licence or detector adequacy. Direct construction of `State` likewise does not certify its diagnostics.

`measure` computes exact diagnostics. Passing a carrier map to `run_tape` recomputes the initial and successor diagnostics, checks that R changes the cochain by a coboundary, and binds topology and positive weights into the tape genesis digest. Without that map, replay checks only the abstract laws. The Lean theorems concern those laws, not an authenticated runtime. The record is:


```
{
  "step_index": 2,
  "before_digest": "sha256 of the canonical before-state",
  "after_digest": "sha256 of the canonical after-state",
  "carrier_before": "two_triangles",
  "carrier_after": "two_triangles",
  "class_before": "ExactRepair",
  "class_after": "ExactRepair",
  "phi_before": "0", "phi_after": "0",
  "gamma_before": "0", "gamma_after": "0",
  "witness": {
    "receipt": false,
    "loss_ledger": false,
    "lift_licence": false,
    "reason": null
```

```
  },
  "control_after": "Open",
  "routing_terminal": null,
  "authority_effect": "None"
}
```

The integration tape chains receipts as $h_{t+1} = \mathrm{SHA256}(t : h_t : \alpha : \mathrm{json}(r_t))$ with canonical JSON (sorted keys, exact rationals as strings, floats refused). Production receipts add a trace identifier, detector and terminalizer versions, protected distinctions, and references to loss ledgers and lift requests. No receipt stores raw user text by default. Tamper evidence assumes the collision resistance of SHA-256 and an independently retained head and length (DDC-I-02). Unanchored replay checks internal consistency only; a rewritten or truncated history can remain internally consistent.

6. Diagnostic and runtime crosswalk

DDC term	Runtime term	Diagnostic meaning	Routing
closure defect	Γ_2	$d_1 x \neq 0$	`Repair` or `Refusal`; never `CheckPassed`
irreducible residual	Φ_1	quotient mass, $\Phi_1 > 0$	`RequiresLift` or residual terminal
exact class	`ExactRepair`	$\Phi_1 = 0$, $\Gamma_2 = 0$	`CheckPassed` within the declared scope
open defect	`ActiveClosure Failure`	$\Gamma_2 > 0$	`Repair` or `Refusal`
stabilized survivor	`Closed Residual`	$\Phi_1 > 0$, $\Gamma_2 = 0$	residual terminal or `Requires Lift`

The four axes stay separate:

`DiagnosticClass` $\neq$ `RoutingTerminal` $\neq$ `ExtensionStatus` $\neq$ `AuthorityStatus.`

Letters are not terminals. `CheckPassed` on the routing axis is not a licence: this package adopts `F_term: REFUSED` as its explicit unlicensed-terminal policy. This is a policy definition, not an application of an external theorem.

7. The laws

The eight candidate laws of the first draft are now step and control laws of a finite-control transition system (Definition 10.1). The comparison columns below refer exclusively to the deposited `engineLaw` and `engine_violations` specifications, retained from repository commit `e4a1214bb3e47570b0f74ecc9117c98f27a26b5e`. Their names are historical. No theorem here establishes parity with a deployed Rust implementation; claims about later runtime changes require a separate pinned conformance check.

1. **Trace typing** (OE010-03). Every letter carries its input class, output class, required witness and refusal condition, and every lawful step satisfies its letter's step law (`Ddc.well_typed_step`).

2. **Terminal absorption** (OE010-10). After `T` no letter is admitted, so `T` is the last letter of a lawful tape (`Ddc.lawful_T_is_last`).
3. **Refusal absorption unless lifted** (DDC-TAPE-01). After `M` only `C` (changed scope), Λ (licensed lift) or `T` may follow.
4. **Lift trigger.** Λ needs a licence and a trigger: the state before it is a `ClosedResidual` or its detector is inadequate. A trigger is not a licence.
5. **Integration requires a receipt.** `I` without a receipt is refused.
6. **Descent requires a loss ledger.** `D` without a loss ledger is refused.
7. **Exact repair discharges.** `E` needs an exact class before it, leaves the carrier unchanged, and leaves $\Phi_1 = \Gamma_2 = 0$ and a zero defect after it. A gauge repair cannot cross classes (Chapter 1), so `E` is never a disguised class change.
8. **Context exposure is not repair.** `C` changes the carrier and never lowers Φ_1 or Γ_2. Hiding an obstruction by re-scoping has to be priced as `D` or Λ.

The step laws, compared with the deposited historical specification:

Letter	Step law	Deposited comparison	Additional paper law
`E`	same carrier; after $\Phi_1 = \Gamma_2 = 0$	yes (`ExactRepairLeftResidual`)	before class exact; zero defect after; adequate detector
`R`	same carrier; Φ_1 unchanged	yes (`ResidualProjectionChangedMass`)	Γ_2 unchanged; adequate detector
`C`	carrier changes	yes (`CarrierUnchangedUnderRestructure`)	Φ_1 and Γ_2 do not fall (Law 8)
`I`	state unchanged	yes (`InertOpMutatedState`)	receipt present (Law 5)
Λ	carrier changes	yes	licence and trigger (Law 4)
`D`	carrier changes	yes	loss ledger (Law 6)
`M`	state unchanged	yes	reason from the closed vocabulary
`T`	state unchanged; $\Gamma_2 = 0$	yes (`TerminalizedOpenResidual`)	adequate detector

Theorem 7.1 (DDC-TAPE-03). Every step admitted by the paper laws satisfies the deposited comparison laws, represented on the abstract state (`engineLaw` in Lean, `engine_violations` in Python). The comparison is defined by the local source bytes; the theorem does not execute Rust or assert correspondence with its current version. Lean: `Ddc.paper_refines_engine`; Python: `test_automaton.py::test_paper_laws_refine_the_engine_laws_on_every_step`.

8. Five minimal demos

Each demo is measured on a declared complex and replayed through the laws by `code/ddc/demos.py`. Unlawful variants are kept as falsifiers. `test_fsm_and_demos.py::test_every_demo_reaches_its_stated_verdict` pins every verdict below.

Demo 1 — boundary cancellation (two triangles sharing an edge). An exact defect is registered, repaired and sealed: `C·R·E·T`, routing `CheckPassed`. A defect with circulation around one face

($\Gamma_2 = 1$) ends in a typed refusal: `C·R·M`. Its falsifier `C·R·E·T` is refused at `E`, because the class before `E` is not exact.

Demo 2 — local green, global red (a filled triangle with value 1 on every edge). Each edge is locally consistent, yet $d_1 x = 3$, so $\Gamma_2 = 3$ and $\Phi_1 = 3$. The lawful tape is `C·R·M`. Appending `T` is refused (`TerminalizedOpenResidual`): a refusal does not license a seal.

Demo 3 — stabilized survivor (a hollow square with a unit defect on one edge; $\Phi_1 = 1$, $\Gamma_2 = 0$). `C·R·T` seals on the residual terminal. A licensed lift that adds capacity away from the cycle, `C·R·Λ·T`, re-measures $\Phi_1 = 1$: the survivor persists, as Chapter 5 proves it must. Without a licence the record is `C·R·M·T` with `REFUSED_REQUIRES_LIFT`. Two falsifiers: a lift without a licence is refused, and a lift that attaches a face bounding the cycle turns the survivor into $\Gamma_2 = 1$, after which `T` is refused.

Demo 4 — detector collapse (a constant detector reports zero on the survivor). The lawful tape is `C·M` with `REFUSED_DETECTOR_INADEQUATE`; `C·M·T` is refused (`BlindDetector`), because a collapsing detector is how false green enters. A licensed detector lift, `C·Λ·T`, re-measures $\Phi_1 = 1$ and seals honestly on the residual terminal.

Demo 5 — descent with loss (a path of four edges carrying $(1, 2, \frac{3}{2}, \frac{1}{4})$, coarsened to two edges). With a loss ledger the tape `D·I·T` books loss $\frac{7}{4}$ and seals. Without one, `D` is refused (`MissingLossLedger`), and the lawful record is `M·T` with `REFUSED_MISSING_LOSS_LEDGER`.

9. Runtime context and verification ceiling

The Rust crate `omega_discrete_deformation_calculus` is related runtime context. Names in `docs/claim_code_map.yaml` are informational pointers, not self-contained evidence of current engine conformance. This deposit proves refinement only of its own historical comparison specification. It supplies no authenticated licence evaluator and no test of the deployed Rust system.

Part II — The eight letters

Chapter	Letter	Central result
1	`E`	gauge repair discharges an exact defect and cannot cross classes
2	`R`	projection keeps the class, with one coordinate per independent cycle
3	`C`	exposure changes the carrier and never lowers the diagnostics
4	`I`	the receipt tape only grows and every letter enters the chain
5	`Λ`	a lift preserves norms; an extending lift cannot exactify a non-exact residual
6	`D`	a descent is lossless exactly when the dropped tail is zero
7	`M`	a refusal leaves the state unchanged and names a reason
8	`T`	the abstract seal needs $\Gamma_2 = 0$ and adequate detection; continuation policy is a separate guard

Part III — Tapes

10. The tape automaton

Definition 10.1. A tape runs from a control state `Open`, `Refused` or `Terminalized` and an abstract data state. A step is admitted when the control admits its letter and the letter's step law holds. `T` moves the control to `Terminalized`, `M` to `Refused`, and `C` or Λ after a refusal back to `Open`. `Terminalized` admits nothing; `Refused` admits only `C`, Λ and `T`. Replay stops at the first unlawful step, which is not recorded. The Python automaton is `code/ddc/automaton.py`; the Lean one is `Ddc.run` in `lean4/Ddc/Law.lean`. They state the same laws. The Lean model represents supplied Φ_1, Γ_2 by $\mathbb{N}$ values; the laws use only equality, order and zero tests, which the non-negative rationals of the Python model share.

11. Tape theorems

Every theorem below is proved in `lean4/` with no axiom outside {`propext`, `Quot.sound`} (`lean4/axiom_report.txt`) and checked on examples in `code/tests/`.

Theorem 11.1 (OE010-03, composition). If replay of t_1 succeeds, replaying $t_1 t_2$ equals replaying t_2 from that successful endpoint. If t_1 fails, the concatenated replay fails as well. The empty tape is the identity (`Ddc.run_append`, `Ddc.lawful_append`, `Ddc.lawful_nil`). Lawful tapes therefore form the morphisms of a category whose objects are control-state pairs.

Theorem 11.2 (OE010-10, terminal absorption). No step is admitted after `T` (`Ddc.terminal_absorbing`, `Ddc.lawful_T_is_last`).

Theorem 11.3 (DDC-TAPE-01, refusal absorption). After `M` only `C`, Λ or `T` is admitted (`Ddc.refusal_absorbing`).

Theorem 11.4 (DDC-TAPE-02, well-typed steps). Every step of a lawful tape satisfies its letter's step law at the state it was taken from (`Ddc.well_typed_step`).

Theorem 11.5 (OE010-04, no authority). Every receipt a replay emits carries `AuthorityEffect.none`, although the type has a second value (`Ddc.authority_effect_none`); the receipt tape only grows (`Ddc.receipts_prefix`).

Theorem 11.6 (OE010-17, no free decrease). If a lawful step lowers Φ_1 or Γ_2, its letter is `D` or Λ (`Ddc.no_free_decrease`).

Proof sketch. `E` starts from $\Phi_1 = \Gamma_2 = 0$; `R` keeps both; `C` cannot lower either; `I`, `M` and `T` leave the state unchanged. □

Theorem 11.7 (DDC-TAPE-04, open obstructions need a priced move). From a state with $\Gamma_2 > 0$ and a non-terminal control, a lawful tape with no `D` and no Λ keeps $\Gamma_2 > 0$ and never terminalizes (`Ddc.open_needs_priced_move`). This is witness dominance at the level of tapes: an open obstruction reaches a seal only through a booked loss or a licensed lift. `test_automaton.py::test_open_obstruction_is_never_sealed_without_a_priced_move` checks it by exhaustive reachability on a 32-state abstraction.

Theorem 11.8 (CREDIT). The mnemonic "CREDIT" names the tape C · R · E · D · I · T; it is only a mnemonic.

- (DDC-TAPE-05) CREDIT is lawful iff each of its six steps satisfies its law (`Ddc.credit_lawful_iff`).
- (DDC-TAPE-06) A lawful CREDIT starts from $\Phi_1 = \Gamma_2 = 0$ (`Ddc.credit_starts_exact`): `E` needs an exact class, `R` keeps the diagnostics and `C` cannot lower them, so the residual was exact before `C`.
- (DDC-TAPE-07) Without a loss ledger at `D` it is not lawful (`Ddc.credit_needs_ledger`).
- (DDC-TAPE-08) It is realizable (`Ddc.credit_realizable`; Python: `demos.credit`, an exact defect on a hollow triangle repaired and then booked down to one edge).

CREDIT is therefore the exact-repair tape with a booked descent. It is not a route for dissolving an obstruction.

Theorem 11.9 (OE010-16, driver soundness). The §12 driver never proposes `T` on an open or blind state, proposes `E` only on an exact class, and proposes Λ only with a Law 4 trigger (`Ddc.driver_never_seals_open`, `Ddc.driver_repairs_only_exact`, `Ddc.driver_lift_trigger`).

12. The §23 driver

The first draft generated tapes with a state machine whose table swapped two classes: it tagged `ActiveClosureFailure` with $\Phi_1 > 0$ and `ClosedResidual` with $\Gamma_2 > 0$. The corrected driver (OE010-16) proposes:

Class of the current state	Letters the driver may propose
`ExactRepair` ($\Phi_1 = 0$, $\Gamma_2 = 0$)	`E` then `T`; `I`
`ClosedResidual` ($\Phi_1 > 0$, $\Gamma_2 = 0$)	`R`, Λ (Law 4), `T` to the residual terminal; `I`
`ActiveClosureFailure` ($\Gamma_2 > 0$)	`R`, `C`, `D`, `M`; `I`; never `T`
detector inadequate	Λ, `M`, `C`; `I`

The four drivers of the first draft read as follows under the laws. `C·R·E·I·T` (autopoietic) is lawful on an exact defect. `C·R·M·T` (refusal) is lawful when the refused state has $\Gamma_2 = 0$; on an open state the lawful record is `C·R·M`. `C·R·Λ·I·T` (self-examination) is lawful from a closed survivor with a licence when the lifted state has $\Gamma_2 = 0$. `D·I·T` (allopoietic) is lawful with a loss ledger when the target has $\Gamma_2 = 0$. The residual-progress guard of the first draft is Theorem 11.6.

13. Open laws

These remain open. Bounded checks are recorded as computed fixtures, never as proofs.

- **Trace equivalence** (DDC-OPEN-01). When do two tapes denote the same handling: same terminal, same diagnostic effect, same receipts, same loss ledger, same carrier endpoint? No equivalence is defined here.
- **Normal forms** (DDC-OPEN-02). Whether finite tapes reduce to forms such as `C·R·E·I·T`, `C·R·M·T`, `C·R·Λ·I·T` and `D·I·T`. No universal normal form is assumed.
- **Detector adequacy in general.** Demo 4 covers the constant detector; the empty carrier, acyclic collapse, shape-only verifiers and all-green or all-open terminalizers remain future fixtures, not results.
- **Separation from rewriting logic**, **lift minimality** and **universal obstruction completeness** are research questions (Part IV).

Composition (Theorem 11.1) and the non-authority invariant (Theorem 11.5), open in the first draft, are now proved.

14. Deformations of the window

Proposition 14.1 (OE010-05). A deformation $d_i \mapsto d_i + \mu_i$ keeps the window a complex iff $(d_1 + \mu_1)(d_0 + \mu_0) = 0$, i.e. iff $d_1\mu_0 + \mu_1 d_0 + \mu_1\mu_0 = 0$ (the Maurer–Cartan condition).

Proof. Expand and use $d_1 d_0 = 0$. □

`cochain.deformation_obstruction` computes the left-hand side exactly; `test_cochain.py::test_maurer_cartan_obstruction_detects_unlawful_deformations` shows a zero deformation passing and a single-entry bump failing.

Part IV — Positioning

Part IV maps the research neighbourhood. Its statements are positioning, not verified literature conclusions; no external citation is upgraded to a result.

15. Research agenda

Six modules steer further work, each recast as a directed attack rather than a validation request.

Module	Question	Relevance to DDC
A. Higher categories and operads	Which structures model finite control loops without unpriced coherence?	may supply composition laws for tapes; not proof that DDC is an operad
B. Sparse ℓ^1 cochains	How stable are quotient obstructions under perturbation?	diagnostic engines for Φ_1 and Γ_2
C. Sheaf obstruction	Which Čech or cellular frameworks separate local consistency from global coherence?	detectors for local-green, global-red failures, once covers are declared
D. Verified rewriting	How can compiler passes be receipt-preserving operations?	concrete software examples for `E`, `D`, `M`, `T`
E. Finite shadows of continua	How can infinite objects be admitted only through finite shadows or oracles?	continuum claims route to `Oracle Needed`
F. Prior art and negative results	Has anyone typed the role a computation plays in an obstruction's life?	tests the novelty statement

Every result from these modules is to be classified as an established finite-hypothesis theorem, a plausible formalization target, an analogy, a likely category error, or unsupported; with the exact construction, the missing adapter data, the smallest demo and the likely counterexample.

16. Prior art and the rewriting-logic objection

Five traditions are closest (DDC-POS-01).

- **Rewriting logic and Maude** is the strongest computational neighbour: it specifies systems by labelled, potentially conditional rules $t \to t'$ modulo equations. Such a specification can include obstruction diagnostics, witness conditions and receipt fields. DDC supplies one declared combination of these choices; this paper proves no separation in expressive power.
- **Constructor Theory** states physics through possible and impossible transformations; it is a philosophical analogy with no operation ledger or cochain quotient.
- **Structural realism** aligns with the representation-first stance but is not mathematical support by itself.
- **Hypergraph rewriting** treats evolution as rule application; its physical claim is empirical, not runtime verification.
- **Coalgebra** gives behaviour and observation semantics, a promising target for trace semantics, without a cochain obstruction calculus.

Adjacent fields supply candidate law sources: trace monoids and abstract rewriting (normal forms, confluence), operational semantics and effect systems (typed letters), provenance semirings and proof-carrying code (receipts), event structures (concurrency), double categories and string diagrams (transport), cellular sheaves and chain homotopy (trace equivalence), discrete Morse theory (cancellation), CEGAR and abstract interpretation (detector adequacy), and equality saturation (repair search).

The sharpest objection is "DDC is rewriting logic with labels". The concrete contribution evaluated here is the selected policy: obstruction diagnostics (§3), typed step guards (§7), chained records (§5), declared licence and ledger conditions (Laws 4 and 6), refusal control (Theorem 11.3), and terminal absorption (Theorem 11.2). Theorem 11.5 proves that emitted receipt fields say `none`; authorization remains outside the model. These choices do not prove that rewriting logic cannot express DDC. A formal translation and a prior-art review of the combination remain open.

17. Novelty statement

Unsafe: "DDC is a theory of everything", "every process is obstruction elimination", "representation is all that exists".

Defensible programme claim (DDC-POS-02): DDC formalizes a finite operation ledger in which every structure-creating, obstruction-exposing, obstruction-eliminating, obstruction-carrying or obstruction-refusing event is a typed step with a replayable record that confers no authorization. What may be distinctive is the combination of six parts:

1. finite cochain diagnostics;
2. typed letters recording what a step did to an obstruction;
3. replayable receipts;
4. proof-assistant and runtime counterparts;
5. machine-readable governance boundaries;
6. a firewall that stops successful representation becoming trusted action.

This stays a programme claim until the prior-art review of Module F is complete.

18. Risks and fences

Risk	Fence
Universality becomes tautology	an obstruction needs a declared carrier, protected distinction, detector, witness or refusal rule, and terminalizer; DDC must say when it returns `NoRule`, `DetectorInadequate` or `OracleNeeded`
"The universe is not creative" needs a negative result	DDC tests whether domains reuse the same letters; it does not prove that no new structure exists
The word "algebra" overclaims	the constructed object is a typed trace calculus with proved tape laws; equivalence and normal forms stay open
Continuum smuggling	continuum objects need finite shadows, completion receipts or oracle interfaces, else `OracleNeeded`
Authority laundering	receipts record conditional decisions; authorization is outside this model. Theorem 11.5 establishes only the constant receipt field
Detector inadequacy	a blind detector cannot seal (`BlindDetector`); authority-sensitive uses need a declared adequacy record

19. Prior-art questions

The standing question for literature search: *has any theory defined an operation alphabet whose symbols classify the role a finite computation played in an obstruction's life, rather than recording that a transition occurred?* Compare against rewriting logic, trace monoids, operational semantics, effect systems, proof-carrying code, provenance semirings, event structures, double categories, string diagrams, cellular sheaves, chain homotopy, deformation complexes, discrete Morse theory, process calculi, higher rewriting, coalgebra, Constructor Theory, hypergraph rewriting and ontic structural realism. For each, classify and return the construction, the finite hypotheses, the missing adapter data, the smallest demo and the likely counterexample. If DDC is rewriting logic with labels, say exactly why; if not, name the minimal formal object that separates them.

20. Final position

DDC is a finite typed trace calculus for obstruction-handling operations. Its product value is that it records what kind of finite operation was performed on an obstruction, not only whether a system passed. Its research value is a possible substrate-independent operation ledger linking cochain obstruction, rewriting, proof-carrying traces, provenance and authority firewalls. The warning stands: do not sell the universe; prove trace equivalence, detector adequacy and the separation from rewriting logic first.

```
DDC tape = replay evidence · DDC receipt = bounded verification data
DDC terminal = routing terminal · DDC authority = None
```

21. Residuals

ID	Residual	Route
R-DDC-01	This version's new mathematics: the tape automaton and Part III	discharged here
R-DDC-02	The historical comparison model's `E` law admits class-crossing edits; the paper requires an exact class before `E`	historical comparison gap; resolved in governance contracts; Rust parity separate
R-DDC-03	The historical comparison model's `R` law does not fix Γ_2	historical comparison gap; resolved in governance contracts; Rust parity separate
R-DDC-04	The historical comparison model does not enforce Law 8 (no scope laundering by `C`)	historical comparison gap; resolved in governance contracts; Rust parity separate
R-DDC-05	The historical comparison model does not check Laws 3–6, the refusal vocabulary or detector adequacy at step level	historical comparison gap; resolved in governance contracts; Rust parity separate
R-DDC-06	Φ_1 needed a dual witness, not only a primal potential	discharged here: `cochain.phi1_certificate`
R-DDC-07	Proposition 3.2 and Theorem 3.3 have written proofs only; mechanizing them needs linear algebra beyond core Lean	Lean: a later version

References

See docs/references.md.

Chapter 1 — Letter E: exact repair

E removes a defect that lies in the image of d_0. It is the *exact* part of the Hodge split, not the co-exact part; the first draft of this chapter had the two reversed.

1.1 Definition

Let $K = (V, E, F)$ be a finite oriented 2-complex with

$$(d_0\varphi)(u \to v) = \varphi(v) - \varphi(u), \qquad (d_1 x)(f) = \sum_{e \in \partial f} [f : e]\, x(e),$$

where $[f : e] \in \{-1, 1\}$ is the orientation of e in the boundary of f. A defect $x \in C^1(K; \mathbb{Q})$ is *exact-repairable* when $x \in \operatorname{im} d_0$, equivalently $\Phi_1(x) = 0$.

Proposition 1.1 (DDC-E-01). An exact defect has a potential φ with $d_0\varphi = x$, unique up to adding one constant on each connected component. `ExactRepairSolver` finds it by breadth-first integration from one gauge vertex per component.

Proof. Two potentials of the same defect differ by ψ with $d_0\psi = 0$, that is, ψ is constant along every edge and hence on every component. □

The repair is $x \mapsto x - d_0\varphi = 0$.

1.2 Step law

E keeps the carrier, starts from an exact class, and leaves $\Phi_1 = \Gamma_2 = 0$ and a zero defect with an adequate detector. The historical comparison model's `verify_step` checks the carrier and the exact class after the step (`ExactRepairLeftResidual`); the class before the step, the zero defect and the detector are checked only by this package (residual R-DDC-02).

1.3 Receipt

`ExactRepairReceipt` records `success`, `input_norm`, `residual_norm`, `gauge_vertex`, `potential`, the repaired defect (`repaired`, all zeros on success), `error` and `authority_effect = "None"`.

1.4 Results

Theorem 1.2 (DDC-E-02). If $d_0\varphi = x$ then $x - d_0\varphi = 0$. Lean: `Ddc.E_gauge_repair_discharges`.

Theorem 1.3 (DDC-E-03). A gauge move $x \mapsto x - d_0\varphi$ changes neither $d_1 x$ nor whether x is exact. Consequently no gauge repair can cross a class: a defect that is closed but not exact, or open, stays so. Lean: `Ddc.E_gauge_preserves_closure`, `Ddc.E_gauge_preserves_exactness`, over an abstract complex with additive d_0, d_1 and $d_1 d_0 = 0$.

Theorem 1.4 (DDC-E-04). A defect with non-zero circulation around any cycle of the declared 1-skeleton is not in $\operatorname{im} d_0$, and E refuses it with `REFUSED_NON_ZERO_CURL`. The name is kept for wire compatibility; its meaning is exactly "not in $\operatorname{im} d_0$". Lean example: `Ddc.E_triangle_open_not_exact` (every edge of a filled triangle carries 1). Tests: `test_exact_repair.py::test_exact_repair_refusal_non_zero_curl`, `test_letter_modules.py::test_exact_repair_refuses_any_cycle_with_circulation`.

Theorem 1.5 (DDC-E-05). Exact repair is idempotent: repairing the repaired (zero) defect again changes nothing. Lean: `Ddc.E_repair_idempotent`.

The orthogonality of the exact part to the harmonic part is Theorem 3.3 of the spine.

1.5 Falsifiers

- Demo 1 applies `E` to an open defect; the step is refused because the class before `E` is not exact (`ExactRepairFromNonExactClass`).
- A class-crossing edit (an open state rewritten to zero) passes the historical comparison model but not the paper (`test_automaton.py::test_engine_checks_less_than_the_paper`).

1.6 Nonclaims

```
E does not claim semantic truth or the correctness of external code or models.
E is a local algebraic gauge adjustment on a declared carrier.
No authorization is conferred.
```

Chapter 2 — Letter R: residual projection

Not every defect can or should be removed. R records what remains, keeping its class, so that an unrepaired residual is never silently discarded or averaged away.

2.1 Definition

Over $\mathbb{Q}$ with the standard inner product the 1-cochains split as $C^1 = \operatorname{im} d_0 \oplus \operatorname{im} d_1^\top \oplus \mathcal{H}^1$ (spine, Theorem 3.3).

Proposition 2.1 (DDC-R-01). Every defect splits uniquely into exact, co-exact and harmonic parts. The first draft named only two parts, which is correct only when there are no faces. Evidence: spine Theorem 3.3, `cochain.hodge_split`, `test_cochain.py::test_hodge_split_is_orthogonal_and_matches_the_classes`.

R's canonical representative subtracts the coboundary of the potential that matches the defect on a spanning forest. What remains is supported on the edges outside the forest: one value per independent cycle.

Proposition 2.2 (DDC-R-06). The register has $b_1 = |E| - |V| + c$ coordinates for a graph with c components. They are the circulations of the defect around the fundamental cycles, and all of them vanish exactly when the defect is exact. The first draft handled a single cycle only. `GraphResidualProjector` covers any finite graph, and `ResidualProjector` keeps the single-cycle case. Tests: `test_letter_modules.py::test_graph_projector_keeps_one_coordinate_per_independent_cycle`, `test_cochain.py::test_a_hollow_cycle_carries_a_closed_residual`.

2.2 Step law

R keeps the carrier, Φ_1 and Γ_2, with an adequate detector. The historical comparison model checks the carrier and Φ_1 (`ResidualProjectionChangedMass`); the Γ_2 clause is paper-only (residual R-DDC-03).

2.3 Receipt

`ResidualProjectionReceipt` records the raw, exact and residual norms, both components, the status (`RESIDUAL_RETAINED` or `EXACT_TENSION_ONLY`), the cycle coordinates and `authority_effect = "None"`.

These norms are unweighted ℓ^1 norms of the recorded vectors, rather than certificates of Φ_1. A fixed forest representative may have norm strictly greater than the quotient minimum. For example, on the increasingly oriented complete graph with vertices $a < b < c < d$ and value -1 on all six edges, the forest rooted at a retains $(-1, -1, -1)$ on bc, bd, cd. Its norm is 3; the potential $(0, -1, -1, -2)$ attains objective 2 and a dual witness certifies $\Phi_1 = 2$. Test: `test_publication_regressions.py::test_forest_register_norm_is_not_the_quotient_minimum`. R preserves the quotient distance because its difference is exact, regardless of whether its representative attains the minimum.

`evaluate_xi_tie_break` compares a secondary exact score only after equal primary diagnostics. It checks the supplied diagnostic values and makes no claim to authenticate or remeasure the candidates.

2.4 Results

Theorem 2.3 (DDC-R-02). The canonical R move is a gauge move, so it keeps Γ_2 and exactness; a lawful R step keeps the class. Lean: `Ddc.R_gauge_move_is_lawful`, `Ddc.R_keeps_diagnostics`, `Ddc.R_keeps_class`.

Proposition 2.4 (DDC-R-03). The defect is conserved edge by edge: the exact component plus the residual component equals the raw defect. Test: `test_letter_modules.py::test_projection_conserves_the_defect`.

Proposition 2.5 (DDC-R-04). Projecting the representative again returns it unchanged. Test: `test_letter_modules.py::test_graph_projector_keeps_one_coordinate_per_independent_cycle`.

Proposition 2.6 (DDC-R-05). Terminalizing an unchanged non-exact state never yields `Check Passed`. A closed residual is sealed only on the residual terminal, and an open one not at all (spine, Theorem 11.7). Test: `test_automaton.py::test_run_tape_stops_at_the_first_unlawful_step_and_chains_receipts`.

2.5 Falsifiers

- An R step that lowers Γ_2 is refused (`ResidualProjectionChangedClosure`).
- An R step that moves Φ_1 is refused by the paper and the historical comparison model (`ResidualProjectionChangedMass`).

The abstract step law compares diagnostics only. Equal norms do not identify quotient classes: on a hollow triangle, (1,0,0) and (−1,0,0) have equal diagnostics but different circulation. `Graph ResidualProjector` subtracts an actual coboundary; `run_tape(..., carriers=...)` checks that stronger condition. A booked descent can subsequently change a non-exact state into an exact one.

2.6 Nonclaims

```
R does not resolve the defect and does not claim that a retained residual will vanish.
R binds unresolved residual mass into the tape. No authorization is conferred.
```

Chapter 3 — Letter C: context exposure

Evaluations are local. C restricts the ambient carrier to a window and exposes every edge that straddles it, so that external dependencies become declared boundary conditions instead of hidden entanglement.

3.1 Definition

For a finite graph $K = (V, E)$ and a window $V_W \subseteq V$, the internal edges are those with both ends in V_W. The boundary interface $\partial_E W$ is the set of edges with exactly one end in V_W. C maps a cochain c to $(c|_W, c|_{\partial_E W})$.

3.2 Step law

C changes the carrier and never lowers Φ_1 or Γ_2 (Law 8). The historical comparison model checks the carrier change (`CarrierUnchangedUnderRestructure`); the monotonicity clause is paper-only (residual R-DDC-04). Hiding an obstruction by choosing a smaller window is scope laundering; it has to be priced as a descent with a loss ledger or a licensed lift.

3.3 Receipt

`ContextExposureReceipt` records the window, the internal and boundary edges, both restricted cochains, the status `CONTEXT_EXPOSED` and `authority_effect = "None"`.

3.4 Results

Proposition 3.1 (DDC-C-01). The internal edges, the boundary edges and the remaining edges partition E. Test: `test_context_exposure.py::test_context_window_disjoint_partition`.

Proposition 3.2 (DDC-C-02). An edit supported on the internal edges leaves the boundary cochain unchanged. Test: `test_letter_modules.py::test_exposure_is_read_only_and_internal_edits_keep_the_boundary`.

Proposition 3.3 (DDC-C-03). Two exposures that agree on every shared edge glue to a unique cochain on the union of their exposed edges; `glue` refuses when a shared edge disagrees. Test: `test_letter_modules.py::test_context_exposure_glues_agreeing_windows_uniquely`.

Proposition 3.4 (DDC-C-04). Exposure is read-only: it returns restrictions and never alters the ambient cochain. Test as for DDC-C-02.

Theorem 3.5 (DDC-C-05). The boundary of an intersection of windows lies in the union of their boundaries. Lean: `Ddc.C_boundary_of_intersection`; test: `test_letter_modules.py::test_boundary_of_an_intersection_lies_in_the_union_of_boundaries`.

Theorem 3.6 (DDC-C-06). A lawful C step changes the carrier and never lowers Φ_1 or Γ_2. Lean: `Ddc.C_is_not_repair`, `Ddc.C_changes_carrier`.

3.5 Falsifiers

- A C step that lowers Γ_2 is refused (`ContextExposureLoweredObstruction`).
- A C step that leaves the carrier unchanged is refused by paper and historical comparison model.

3.6 Nonclaims

```
C does not claim that a window is complete or that unexposed values are trivial.
C is not repair. No authorization is conferred.
```

Chapter 4 — Letter I: integration

No progress counts until it is committed. I appends a receipt to an append-only, hash-chained tape and changes nothing else.

4.1 Definition

The tape at step t is $T_t = [r_0, \ldots, r_{t-1}]$. With $h_0 = \mathrm{SHA256}(\text{genesis root})$, the receipt for letter $\alpha \in \{E, R, C, I, \Lambda, D, M, T\}$ is chained as

$$h_{t+1} = \mathrm{SHA256}(t : h_t : \alpha : \mathrm{json}(r_t)).$$

Every letter enters the chain, I included; the first draft's letter set left I out. The index, the previous head (64 hex digits) and the letter cannot contain the separator, so the preimage parses uniquely. Payloads are canonical JSON with exact rationals written as strings, and floats are refused.

4.2 Step law

I leaves the state unchanged and carries a receipt (Law 5). The historical comparison model checks the unchanged state (`InertOpMutatedState`); the receipt clause is paper-only (residual R-DDC-05).

4.3 Receipt

`TapeEntry` records `index`, `prev_hash`, `letter`, the canonical `payload` and `current_hash`.

4.4 Results

Theorem 4.1 (DDC-I-01). The tape only grows: a tape is a prefix of every later tape, and the receipts of a replay are a prefix of the receipts of any extension of it. Lean: `Ddc.I_commit_prefix`, `Ddc.I_receipts_prefix`.

Proposition 4.2 (DDC-I-02). A broken hash or predecessor link is located by `first_broken_index`. Replacing a final entry and rehashing it, rewriting a suffix, or truncating the tape can leave a consistent chain. `verify_integrity(checkpoint)` compares with an independently retained genesis hash, head hash and length; under SHA-256 collision resistance it detects these changes. A checkpoint obtained from the suspect tape supplies no authentication. A final checkpoint detects a mismatch but does not locate the original first alteration. Tests: `test_review_regressions.py::test_tape_checkpoint_detects_rehashed_replacement_and_truncation`.

Proposition 4.3 (DDC-I-03). Replaying the same steps from the same state reproduces every state and the same head hash. Tests: `test_automaton.py::test_replay_is_deterministic`, and the pinned heads in `test_fsm_and_demos.py`.

Proposition 4.4 (DDC-I-04). A step that fails its law is not committed: replay stops before recording it. Lean: `Ddc.I_needs_receipt`; test: `test_automaton.py::test_run_tape_stops_at_the_first_unlawful_step_and_chains_receipts`.

4.5 Falsifiers

- An I step without a receipt is refused (`MissingReceipt`).
- An I step that alters the state is refused by paper and historical comparison model.

4.6 Nonclaims

```
I does not claim semantic authority or that committed steps are optimal.
I provides deterministic commitment; historical integrity needs a trusted checkpoint.
No authorization is conferred.
```

Chapter 5 — Letter Λ: structural lift

When an obstruction cannot be handled on the current carrier, ad hoc expansion would launder authority. Λ is licensed carrier expansion: it needs a licence, and it needs a trigger that makes a lift the right move. A trigger is not a licence.

5.1 Definition

The lift pads with zeros: $\iota : \mathbb{Q}^n \hookrightarrow \mathbb{Q}^{n+k}$, $\iota(x)_i = x_i$ for $i \leq n$ and 0 otherwise.

5.2 Step law

Λ changes the carrier, carries a licence, and starts from a `ClosedResidual` or from an inadequate detector (Law 4). The historical comparison model checks the carrier change; the licence and the trigger are paper-only (residual R-DDC-05). The first draft did not state the trigger.

5.3 Receipt

`LiftReceipt` records the initial and extended dimensions, the licence, the lifted vector, the trigger, `success`, `error` (`REFUSED_UNLICENSED_LIFT`, `REFUSED_LIFT_TRIGGER` or `INVALID_NEGATIVE_EXPANSION`) and `authority_effect = "None"`.

The auxiliary `FiniteLambdaLedger` stores exact nonnegative diagnostic rows and checks endpoint matching, adjacency, non-final `is_descent` flags and the final `is_valid` flag. Both flags must be actual booleans; the row sequence is snapshotted. They remain assumptions, rather than checked preservation or licence certificates. This consistency gate does not execute the external Rust `require_descent_step` or `require_valid` methods and establishes no current contract conformance. It also does not impose a resolved final endpoint on a nonempty ledger; that condition needs a separate policy.

5.4 Results

Theorem 5.1 (DDC-L-01). Zero-padding preserves the ℓ^1 norm, and every inner product among old vectors. Lean: `Ddc.L_pad_preserves_l1`; test: `test_letter_modules.py::test_lift_trigger_law_and_norm_preservation`.

Proposition 5.2 (DDC-L-02). A lawful Λ carries a licence and a trigger. Lean: `Ddc.L_needs_licence_and_trigger`.

Proposition 5.3 (DDC-L-03). The new coordinates are orthogonal to the image of ι, since their supports are disjoint.

Proposition 5.4 (DDC-L-04). An unlicensed or untriggered lift fails closed and returns the original vector. Tests: `test_structural_lift.py::test_unlicensed_lift_refusal`, `test_letter_modules.py::test_lift_trigger_law_and_norm_preservation`.

Theorem 5.5 (DDC-TAPE-09). Let K_0 sit in K_1 with restriction maps that commute with d_0, and let x' on K_1 restrict to x on K_0. If x' is exact on K_1, then x is exact on K_0. So a non-exact residual stays non-exact under every lift that extends it. Lean: `Ddc.L_cannot_exactify`.

Proof. If $x' = d_0\varphi$ then $x = \rho_1(d_0\varphi) = d_0(\rho_0\varphi)$. □

Demo 3 shows both halves. A lift that adds capacity away from the cycle keeps $\Phi_1 = 1$, and the survivor is sealed on the residual terminal. A lift that attaches a face bounding the cycle turns the survivor into an open defect ($\Gamma_2 = 1$) that `T` may not seal.

5.5 Falsifiers

- A lift without a licence is refused (`UnlicensedLift`).

- A lift from an exact or an open, adequately detected state is refused (`LiftWithoutTrigger`).

5.6 Nonclaims

`Λ does not grant unbounded or continuous expansion and does not resolve a survivor.`
`Λ requires an explicit discrete licence. No authorization is conferred.`

Chapter 6 — Letter D: descent

Moving to a coarser carrier discards distinctions. D does not claim that the coarse state is faithful; it books exactly what was lost.

6.1 Definition

With $\pi : \mathbb{Q}^{n+k} \to \mathbb{Q}^n$ keeping the first n coordinates and ι the zero-padding inclusion, the loss vector is $x - \iota(\pi(x))$ (the dropped tail) and the loss scalar is its ℓ^1 norm $L(x) = \sum_{i>n}|x_i|$.

6.2 Step law

D changes the carrier and carries a loss ledger (Law 6). The historical comparison model checks the carrier change; the ledger clause is paper-only (residual R-DDC-05). D is one of the two letters that may lower Φ_1 or Γ_2 (spine, Theorem 11.6), which is why its loss must be booked.

6.3 Receipt

`DescentReceipt` records the source and target dimensions, the coarse vector, the loss vector, the loss scalar, `is_lossless`, the status (`LOSSLESS_DESCENT` or `DESCENT_WITH_LOSS`) and `authority_effect = "None"`. `ledger_entry` is the ledger a D step carries.

6.4 Results

Proposition 6.1 (DDC-D-01). The loss is non-negative (a sum of absolute values; in Lean a natural number).

Theorem 6.2 (DDC-D-02). The descent is lossless exactly when every dropped coordinate is zero, that is, when $x \in \iota(\mathbb{Q}^n)$. Lean: `Ddc.D_lossless_iff`, `Ddc.l1_eq_zero_iff`.

Theorem 6.3 (DDC-D-03). Projecting twice is projecting once. Lean: `Ddc.D_projection_idempotent`.

Proposition 6.4 (DDC-D-04). A non-zero loss can never produce a `LOSSLESS_DESCENT` receipt, and a lawful D carries its ledger. Lean: `Ddc.D_needs_ledger`; tests: `test_descent.py::test_descent_with_positive_loss`, `test_letter_modules.py::test_descent_books_its_ledger`.

6.5 Falsifiers

- Demo 5 without a ledger: `D` is refused (`MissingLossLedger`), and the lawful record is `M·T` with `REFUSED_MISSING_LOSS_LEDGER`.
- CREDIT without a ledger at D is not lawful (spine, Theorem 11.8).

6.6 Nonclaims

```
D does not claim lossless projection unless the loss is zero.
D does not permit unrecorded loss of distinctions. No authorization is conferred.
```

Chapter 7 — Letter M: typed refusal

The ability to refuse is co-equal with the ability to deform. M is a lawful refusal outcome: it leaves the state unchanged, names its reason and moves control to `Refused`. It does not seal the tape; a later C, Λ or T still needs its own law and control check.

7.1 Definition

A request is a letter, a locus, an alleged witness and a cost. Grounding is checked at intake, before admission; witness and budget are checked on the tape as M.

7.2 Step law

M leaves the state unchanged and carries a reason from the closed vocabulary below. The historical comparison model checks the unchanged state (`InertOpMutatedState`). It carries its reason as a free string (`dda_alphabet::StepOutcome::RefusedToExecute`), so this vocabulary is owned by the paper (residual R-DDC-05). After M, only `C`, Λ or `T` may follow (spine, Theorem 11.3).

7.3 The refusal vocabulary

Intake refusals are recorded before admission and never enter the tape (`enters_ddc_trace` is false):

Code	Meaning
`REFUSED_UNTYPED`	the request or its defect is not typed on the declared carrier
`REFUSED_UNGROUNDED_LOCUS`	the locus is not in the declared carrier
`ADAPTER_INADEQUATE`	the adapter cannot represent the request

In-trace reasons carried by M:

Code	Raised when
`REFUSED_INVALID_WITNESS`	the witness is missing or does not verify
`REFUSED_BUDGET_EXHAUSTED`	the cost exceeds the remaining budget
`REFUSED_NON_ZERO_CURL`	E was requested for a defect not in $\operatorname{im} d_0$
`REFUSED_UNLICENSED_LIFT`	Λ was requested without a licence
`REFUSED_LIFT_TRIGGER`	Λ was requested without a Law 4 trigger
`REFUSED_MISSING_LOSS_LEDGER`	D was requested without a loss ledger
`REFUSED_MISSING_RECEIPT`	I was requested without a receipt
`REFUSED_REQUIRES_LIFT`	a closed survivor needs a lift that is not licensed
`REFUSED_DETECTOR_INADEQUATE`	the detector cannot certify the diagnostics
`REFUSED_OPEN_OBSTRUCTION`	an open obstruction ($\Gamma_2 > 0$) cannot be sealed

The first draft used both `MissingLossLedger` and SCREAMING_CASE codes, and listed the ungrounded locus both as an intake refusal and as an M reason. It is an intake refusal.

7.4 Results

Theorem 7.1 (DDC-M-01). A lawful M leaves the state unchanged and names a reason, and it moves the control to `Refused`. Lean: `Ddc.M_preserves_state`, `Ddc.M_refuses`.

Proposition 7.2 (DDC-M-02). An ungrounded locus is refused at intake and never enters the tape. Test: `test_letter_modules.py::test_ungrounded_locus_is_an_intake_refusal_that_never_enters_the_tape`.

Proposition 7.3 (DDC-M-03). A missing or invalid witness yields `REFUSED_INVALID_WITNESS`. Test: `test_typed_refusal.py::test_refusal_invalid_witness`.

Proposition 7.4 (DDC-M-04). A cost above the remaining budget yields `REFUSED_BUDGET_EXHAUSTED`. Test: `test_typed_refusal.py::test_refusal_budget_exhausted`.

7.5 Falsifiers

- An M step without a reason, or with a reason outside the vocabulary, is refused (`MissingRefusalReason`).
- An M step that alters the state is refused by paper and historical comparison model.

7.6 Nonclaims

```
M is not a system failure. It is a typed, lawful refusal that preserves state.
No authorization is conferred.
```

Chapter 8 — Letter T: terminalization

A recursive evaluation needs a halting law that prevents infinite expansion and self-certification. T closes a tape on a routing terminal when the declared terminal policy allows it, subject to the sealing conditions below.

8.1 Definition

`CanContinue_n` is the conjunction of the nine conjuncts named by the historical comparison model (`omega_discrete_deformation_calculus::oeprmalg2::CONTINUATION_CONJUNCTS`):

1. an outstanding obligation exists;
2. a higher obligation witness exceeds it;
3. the next carrier is declared;
4. the adapter is sound (its loss ledger is booked);
5. the detector is adequate;
6. any lift is minimal and licensed;
7. the old skeleton is preserved;
8. budget is available;
9. no GTLA continuation refusal was issued.

The first draft's three-condition form (positive budget, unresolved residual, strict progress) is a weakening. It is conjuncts 8 and 1, plus the progress guard of spine Theorem 11.6.

8.2 Step law

T leaves the state unchanged, requires $\Gamma_2 = 0$, and requires an adequate detector. The historical comparison model checks the unchanged state and $\Gamma_2 = 0$ (`TerminalizedOpenResidual`); the detector clause is paper-only (residual R-DDC-05). After T nothing is admitted.

8.3 Verdicts

The abstract Lean T law does not include `CanContinue_n`; its theorem concerns only the conjunction itself. `TerminalizationEvaluator.evaluate` accepts optional `continuation_facts`. After the open/blind-state guards, if all nine hold it returns `CAN_CONTINUE`; if one fails, an open or blind state still cannot seal, and a closed positive residual receives `HALTED_CONTINUATION_POLICY`. These booleans are supplied assumptions, not independently verified facts.

Without `continuation_facts`, the separate residual/budget/lookback policy returns one of the following. Its simple-policy tests do not establish the nine-fact policy for a deployed system:

Verdict	When	Routing
`SEAL_REFUSED_OPEN_OBSTRUCTION`	$\Gamma_2 > 0$	not terminal; the tape ends in M
`SEAL_REFUSED_DETECTOR_INADEQUATE`	blind detector	not terminal; the tape ends in M
`LOCAL_GREEN_FOR_SCOPE`	zero residual	`CheckPassed` inside the declared scope
`HALTED_OUT_OF_BUDGET`	closed residual, budget spent	residual terminal
`HALTED_STAGNANT_PROGRESS`	no progress over the lookback window	residual terminal

Every receipt records `authority_terminal = "F_term:REFUSED"`. A local green licenses nothing: refusal on the authorization axis is an explicit package policy, not an inferred external theorem. The first draft emitted a terminal verdict for an out-of-budget open obstruction; that verdict now routes to M (`REFUSED_OPEN_OBSTRUCTION`).

8.4 Results

Proposition 8.1 (DDC-T-01). If every charged step costs at least $\varepsilon > 0$ from an initial budget B_0, at most $\lfloor B_0/\varepsilon \rfloor$ charged steps occur. This is a conditional theorem with a one-line proof: the budget is non-negative and falls by ε per step.

Proposition 8.2 (DDC-T-02). Under the simple policy, a zero residual with $\Gamma_2 = 0$ and an adequate detector yields `LOCAL_GREEN_FOR_SCOPE` and records the unlicensed authority terminal. Tests: `test_terminalization.py::test_terminal_local_green`, `test_letter_modules.py::test_terminal_evaluator_refuses_to_seal_open_or_blind_states`.

Proposition 8.3 (DDC-T-03). A non-decreasing residual over the lookback window halts with `HALTED_STAGNANT_PROGRESS`. Test: `test_terminalization.py::test_terminal_stagnant_`

`progress`.

Theorem 8.4 (DDC-T-04). T seals: the control becomes `Terminalized` and admits nothing. Lean: `Ddc.T_seals`, `Ddc.terminal_absorbing`.

Theorem 8.5 (DDC-T-05). A lawful T leaves the state unchanged, has $\Gamma_2 = 0$ and an adequate detector, and lands on the exact or the closed class. Lean: `Ddc.T_never_seals_open`, `Ddc.T_routing`.

Theorem 8.6 (DDC-T-06). `CanContinue_n` holds exactly when all nine conjuncts hold, and each conjunct is a strict boolean, never an integer. Lean: `Ddc.canContinue_iff`; test: `test_letter_modules.py::test_can_continue_is_the_conjunction_of_the_nine_named_conjuncts`.

8.5 Falsifiers

- Demo 2 appends `T` to an open refusal: refused (`TerminalizedOpenResidual`).
- Demo 4 appends `T` under a blind detector: refused (`BlindDetector`).

8.6 Nonclaims

T does not mint global authority and never emits an unscoped green verdict. A terminal receipt records a conditional terminal-policy decision. No authorization is conferred.

Appendices — Discrete Deformation Calculus v1.0.3

Part of: Discrete Deformation Calculus v1.0.3

The appendices hold candidate adapters, observer notes and worked trajectories. Nothing here is part of the core calculus unless its ledger row says so.

A. Candidate adapter: ZX diagrams to cochains

> **Fence.** A candidate adapter from planar ZX diagrams to finite cochains. It is not core DDC and asserts no quantum or physical result (DDC-OPEN-03).

A finite ZX diagram gives a graph carrier: spiders with a type Z or X and a phase θ_v are vertices, oriented wires with a phase discrepancy ω_e are edges, and closed plaquettes are faces. The candidate coboundary is colour-relative:

$$(d_0\theta)_{u\to v} = \theta_v - \theta_u - H_{u,v}, \qquad H_{u,v} = \begin{cases} 0 & \text{same type,} \\ \pi/2 & \text{different type,} \end{cases}$$

with d_1 summing signed discrepancies around each plaquette. Two obligations are open. First, d_0 with a non-zero H is affine, not linear, so $d_1 \circ d_0 = 0$ must be proved or refused for the declared H. Second, phases are real, so an exact treatment needs a finite shadow over $\mathbb{Q}$ or a declared oracle. Spider fusion $Z(\alpha) \circ Z(\beta) \Rightarrow Z(\alpha + \beta)$ would read as E when a local θ with $\theta_v - \theta_u = \omega_e$ exists, giving the candidate tape `C·R·E·I·T`.

B. Observer gauges and dual witnesses

> **Fence.** An observer-gauge note. It is not core DDC, and it leaves $\mathbb{Q}$ where it takes a square root (DDC-OPEN-04).

For a two-component defect $\omega = (1, 1)$ on one edge of a hollow six-cycle, a coordinate-separable observer reads $\|(1,1)\|_1 = 2$, and a rotation-invariant observer reads $\|(1,1)\|_2 = \sqrt{2}$. The ratio $\sqrt{2}$ is specific to this defect and these gauges; for (1,0) the ratio is 1. A feasible dual flow must have zero divergence. For the separable gauge, the vector (1,1) circulates on every consistently oriented cycle edge; only its pairing with the one-edge defect is concentrated on that edge. The rotation-invariant gauge similarly uses the constant unit flow $(1, 1)/\sqrt{2}$ on every edge. A one-edge dual vector would violate flow conservation. Formalizing the finite dual certificate, with its Fenchel–Rockafellar hypotheses named, is open.

C. Worked trajectories

C.1 Trajectory recording (OE010-06). A trajectory maps concrete tool and human actions into cochain classes, and cochains back into readable evidence, with $d_k \circ d_{k-1} = 0$ on the window. This is a positioning statement. The recorder cited by the first draft is not part of this deposit, and no claim here depends on it.

C.2 Projection-context congruence (OE010-07). Following the coalgebraic partition refinement of Birkmann et al. (arXiv:2204.06248), signatures $\mathrm{sig}_\pi = F\pi \circ c$ split states until a representation p and every context continuation agree. When $p(x) = p(y)$ but some context c separates them,

the pair and its separating word witness a *returning projection*. This is positioning. A related implementation exists elsewhere in the repository, outside this deposit, and is not relied on here.

C.3 Newton–Leibniz as a tape (OE010-08). Integrate $f(x) = x$ on $[0, 10]$ with ten left-endpoint steps, exactly over $\mathbb{Q}$. The carrier is a path with vertices $0, \ldots, 10$. The measured increments are k (left sum 45). The exact increments are d_0F for $F(k) = k^2/2$, namely $k + \frac{1}{2}$. The defect is $\frac{1}{2}$ on every edge, total 5. A path has no cycles, so every defect on it is exact. The tape `C·R·E·I·T` declares the grid, registers the defect, repairs it to d_0F, commits and seals. The integral is the telescoping sum $F(10) - F(0) = 50$: on a path, the discrete fundamental theorem of calculus says that summing a coboundary returns the difference of the potential at the endpoints. `demos.newton_leibniz` computes these numbers and the tape's receipt head; `test_fsm_and_demos.py::test_newton_leibniz_is_exact` pins both. No hash is copied into this text.

D. Spectral sequences, skills and operads

Fence. These are programme definitions, not theorems. No code in this deposit implements them, and the first draft's references to such code were dead.

- **Self-maintenance mode** (OE010-11). A tape that re-examines its own detector through `C·R·Λ·I·T` would relate to the E_∞ page of a filtration spectral sequence. Open.
- **Skill objects** (OE010-12). A *skill* is proposed to be a residual class that survives every licensed attachment in a growth filtration, bound to a receipt. Definition proposal; open.
- **Meta-skills** (OE010-13). A meta-skill is proposed to be a class that persists across a family of carriers. Open.
- **Spectral sequence on a minimal complex** (OE010-14). Attaching a 2-cell σ with nonzero pairing $\omega(\partial\sigma) \neq 0$ changes the question: ω is no longer closed, so the class leaves H^1 as an open face defect ($\Gamma_2 > 0$), and it is not repaired. Demo 3's falsifier shows this directly, and Theorem 5.5 shows that no lift makes a survivor exact under its commuting-restriction hypotheses. Support containment alone is insufficient: two triangle circulations $+1$ and -1 cancel on a face traversing both cycles (`test_review_regressions.py::test_attaching_a_face_must_pair_nontrivially_with_the_residual`). A correct page-by-page account over a declared filtration is open.
- **Coloured operads of skills** (OE010-15). Composition $\gamma \in \mathcal{T}(c_1, \ldots, c_n; d)$ with loss-ledger accumulation $L_{\gamma\circ(\delta_i)} = L_\gamma \cup \bigcup_i \gamma_*^{(i)}(L_{\delta_i})$ is a design sketch. The associativity and unit laws of such an operad are open, and composition mints no authority.

References and their role

External literature supplies definitions and context; none of these references authenticates a DDC runtime or supplies a licence. Local companion links are pinned to repository commit e4a1214bb3e47570b0f74ecc9117c98f27a26b5e.

1. J. H. Carroll. *Local Green, Global Red I: Evaluated Path Kernels for Composed Agent Reliability*, v1.2.3 (2026). Pinned companion source. Context for the local/global distinction; no DDC proof depends on it.
2. J. H. Carroll. *Finite Obstruction Calculus: Exact Quotient Diagnostics, Dual Witnesses, Two-Stage Coherence, and Transport*, v3.2.0. Pinned companion source. Context for §3; the DDC graph formulas and arguments are supplied locally.
3. J. H. Carroll. *Six jobs, not one machine: obstruction, standing, transport, comparison, shadow, leftover*, v3.2.0 (2026). Pinned companion source. Background to the authorization distinction. DDC's refusal terminal is an explicit local policy, not an application of an unstated external theorem.
4. F. Birkmann, H.-P. Deifel and S. Milius. *Distributed Coalgebraic Partition Refinement*. TACAS 2022, LNCS 13244, 159–177. DOI 10.1007/978-3-030-99527-0_9, arXiv:2204.06248. Positioning for Appendix C.2; no implementation is imported.
5. J. Meseguer. *Conditional rewriting logic as a unified model of concurrency*. Theoretical Computer Science 96(1), 73–155 (1992). DOI 10.1016/0304-3975(92)90182-F. Prior-art context for §16; separation from rewriting logic remains open. Maude project overview and documentation. The framework can represent other operational specifications; DDC's chosen guards and receipt fields do not establish an expressivity separation.
6. D. Deutsch and C. Marletto. *Constructor theory of information*. Proceedings of the Royal Society A 471, 20140540 (2015). DOI 10.1098/rspa.2014.0540, arXiv:1405.5563. Possible/impossible transformations are a comparison in §16, not a physics claim.
7. J. J. M. M. Rutten. *Universal coalgebra: a theory of systems*. Theoretical Computer Science 249(1), 3–80 (2000). CWI publication record, DOI 10.1016/S0304-3975(00)00056-6. Coalgebraic context for signatures and transition systems in Appendix C.2.
8. B. Coecke and A. Kissinger. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press (2017), ISBN 978-1-107-10422-8. Publisher record. Diagrammatic context only; Appendix A's proposed adapter remains unproved.
9. L.-H. Lim. *Hodge Laplacians on Graphs*. SIAM Review 62(3), 685–715 (2020). DOI 10.1137/18M1223101, arXiv:1507.05379. Linear-algebra background for the decomposition stated and proved in §3.
10. The Lean project. Lean 4 reference manual. The executable toolchain is pinned to v4.34.0 in lean4/lean-toolchain; the axiom report records the actual compiler version and commit.